>_ MCMP113

Introduction to Basic Algorithms and Programming

Week 5

Decision Structures: if-else, else-if and switch-case

Choosing what runs · else-if ladders · switch · input validation with TryParse

```
> dotnet run
Enter an integer: 7
7 is odd.
Check with ?: -> odd
> dotnet run
Enter an integer: 10
10 is even.
Check with ?: -> even
> _
```

Asst. Prof. Hasan Oğuz, PhD
Istanbul Okan University · Vocational School
2026–2027 Fall Semester

>_ LEARNING OBJECTIVES

By the End of This Week You Can …

Write if / if-else

Run a block only when a bool condition is true; draw the matching flowchart.

Build else-if ladders

Choose among many ranges (grades, BMI, prices) in the correct order.

Nest decisions

Combine nested ifs with && and ||; avoid the dangling-else trap.

Use switch

Multi-way choice on int, char or string: case, break, default, stacked labels.

Validate input

Use int.TryParse / double.TryParse plus range checks instead of crashing.

Test every branch

Choose boundary inputs so that every path runs at least once.

From true / false to Choosing a Path

C# expression (x = 7)	Value	Week 4 idea
`x > 5`	`true`	relational operator
`x % 2 == 0`	`false`	remainder + equality
`x >= 0 && x <= 100`	`true`	range test with &&
`x < 0 || x > 100`	`false`	outside-range test with ||
`!(x == 7)`	`false`	negation with !

Week 5: the bool decides what runs

```
int x = 7;
if (x % 2 == 0)
{
    Console.WriteLine("even");
}
else
{
    Console.WriteLine("odd");
}
```

Sequence

Weeks 3–4 · one step after another

Selection

Week 5 · TODAY · choose a path

Repetition

Weeks 6–7 · repeat a block



PART 1

The if Statement

One condition, one or two paths

if: Run a Block Only When the Condition Is True

Syntax

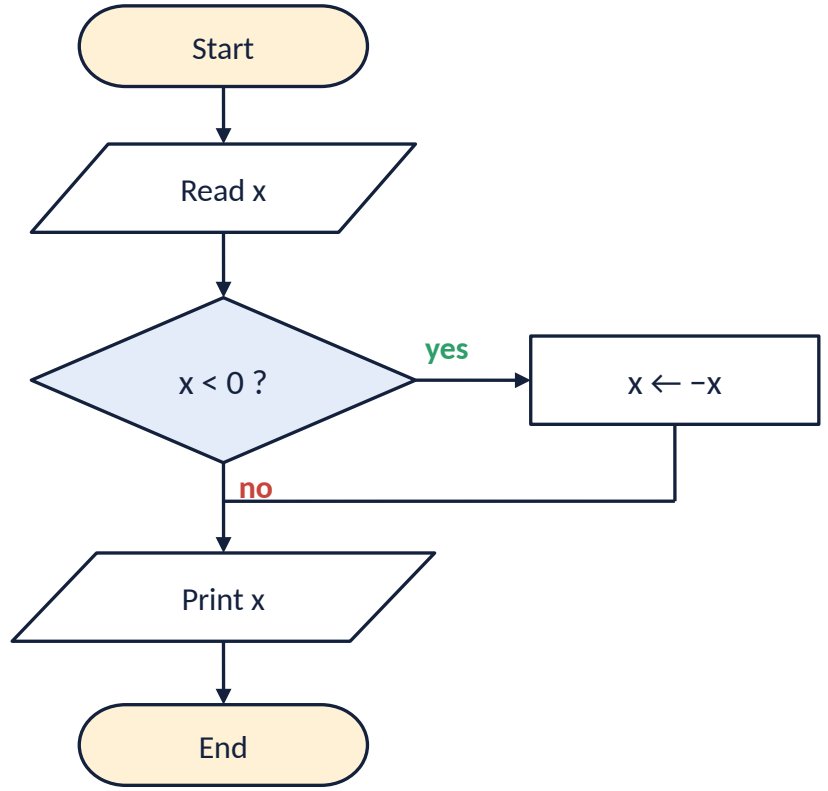
```
if (condition)          // condition: a bool
{
    // runs only when condition is true
}
```

Example: absolute value

```
Console.Write("Enter a number: ");
int x = int.Parse(Console.ReadLine());
if (x < 0)
{
    x = -x;   // runs only when x is negative
}
Console.WriteLine($"Absolute value: {x}");
```

Console

```
Enter a number: -12
Absolute value: 12
```



No else: when the condition is false, the block is simply skipped.

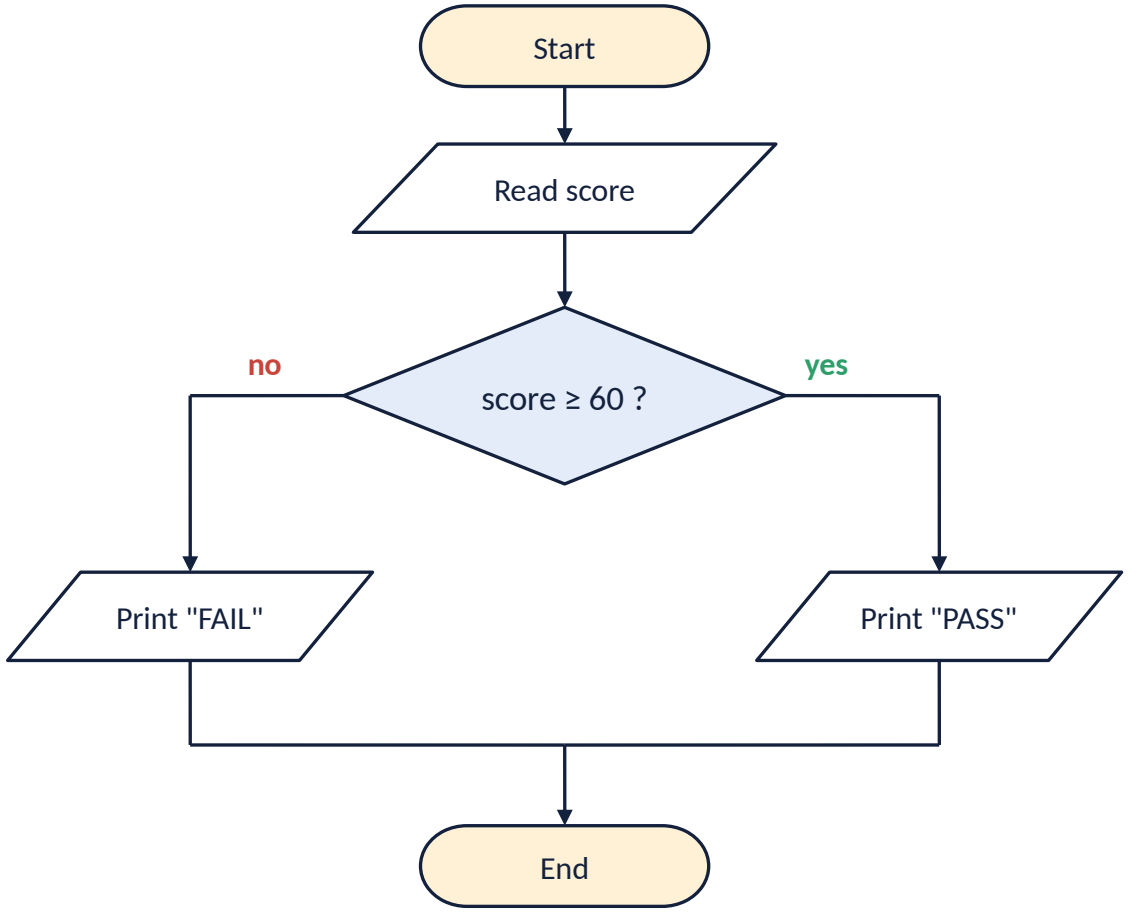
if-else: Exactly One of Two Blocks Runs

Pseudocode

```
START
    READ score
    IF score ≥ 60 THEN
        PRINT "PASS"
    ELSE
        PRINT "FAIL"
    END IF
END
```

C# (core of 05_01_PassFail)

```
if (score >= PassMark)
{
    Console.WriteLine($"Score {score}: PASS");
}
else
{
    Console.WriteLine($"Score {score}: FAIL");
}
```



Even or Odd, and the Conditional Operator ?:

Program.cs (excerpt)

```
// Version 1: if-else chooses between two ACTIONS.
// Test "== 0", not "== 1": in C#, -7 % 2 is -1.
if (n % 2 == 0)
{
    Console.WriteLine($"{n} is even.");
}
else
{
    Console.WriteLine($"{n} is odd.");
}

// Version 2: the conditional operator ?: chooses between two VALUES.
string kind = (n % 2 == 0) ? "even" : "odd";
Console.WriteLine($"Check with ?: -> {kind}");
```

Test **`== 0`**, not **`== 1`**: in C#, `-7 % 2` is **`-1`**.

Console

```
Enter an integer: -7
-7 is odd.
Check with ?: -> odd

Enter an integer: 10
10 is even.
Check with ?: -> even
```

Conditional operator ?:

`cond ? valueIfTrue : valueIfFalse`

if-else chooses between two ACTIONS.
?: chooses between two VALUES — handy inside an assignment or an interpolated string.

Indentation Does Not Make a Block — Braces Do

Missing braces

```
int balance = 500;
int amount = 150;
if (amount > balance)
    Console.WriteLine("Insufficient funds.");
    Console.WriteLine("Transaction cancelled.");
Console.WriteLine("Goodbye.");
```

Console

```
Transaction cancelled.
Goodbye.
```

Only the FIRST statement belongs to the if.

Stray semicolon

```
int temperature = 15;
if (temperature > 30);
{
    Console.WriteLine("It is hot!");
}
Console.WriteLine("Done.");
```

Console

```
It is hot!
Done.
```

`warning CS0642`: Possible mistaken empty statement



Course rule: always use braces { }, even for a single statement. Then what you see (indentation) and what runs are the same.

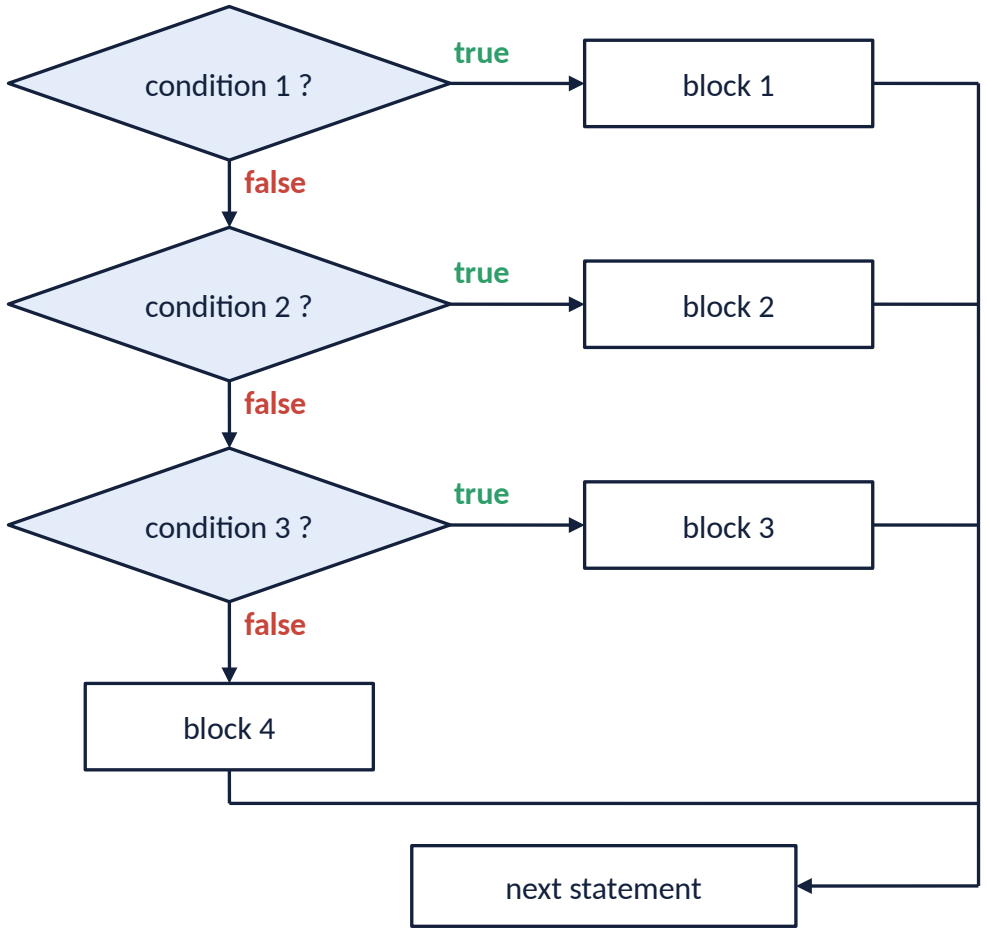


PART 2

else-if Ladders and Nested Decisions

Many ranges, many conditions — one answer

The else-if Ladder: First True Condition Wins




```
if (condition1)
{
    // block 1
}
else if (condition2)
{
    // block 2
}
else if (condition3)
{
    // block 3
}
else
{
    // block 4: none was true
}
```

Exactly ONE block runs. Later conditions are not even tested.

Order Matters: the Ladder Stops at the First Match

WRONG order (shortened scale)

```
string letter;
if (score >= 60)
{
    letter = "DD";
}
else if (score >= 80)
{
    letter = "BB";   // never reached!
}
else if (score >= 90)
{
    letter = "AA";   // never reached!
}
else
{
    letter = "FF";
}
```

score	wrong order	correct order
95	DD	AA
85	DD	BB
72	DD	DD
60	DD	DD
59	FF	FF



Fix: test the highest limit first: >= 90, then >= 80, then >= 60, else FF.

A **switch expression** in this wrong order does not even compile (error CS8510).

Letter Grades with an else-if Ladder

Score	Letter	Point
90 – 100	AA	4.0
85 – 89	BA	3.5
80 – 84	BB	3.0
75 – 79	CB	2.5
70 – 74	CC	2.0
65 – 69	DC	1.5
60 – 64	DD	1.0
0 – 59	FF	0.0

Example scale for our exercises — check the university regulation for the official cut-offs.

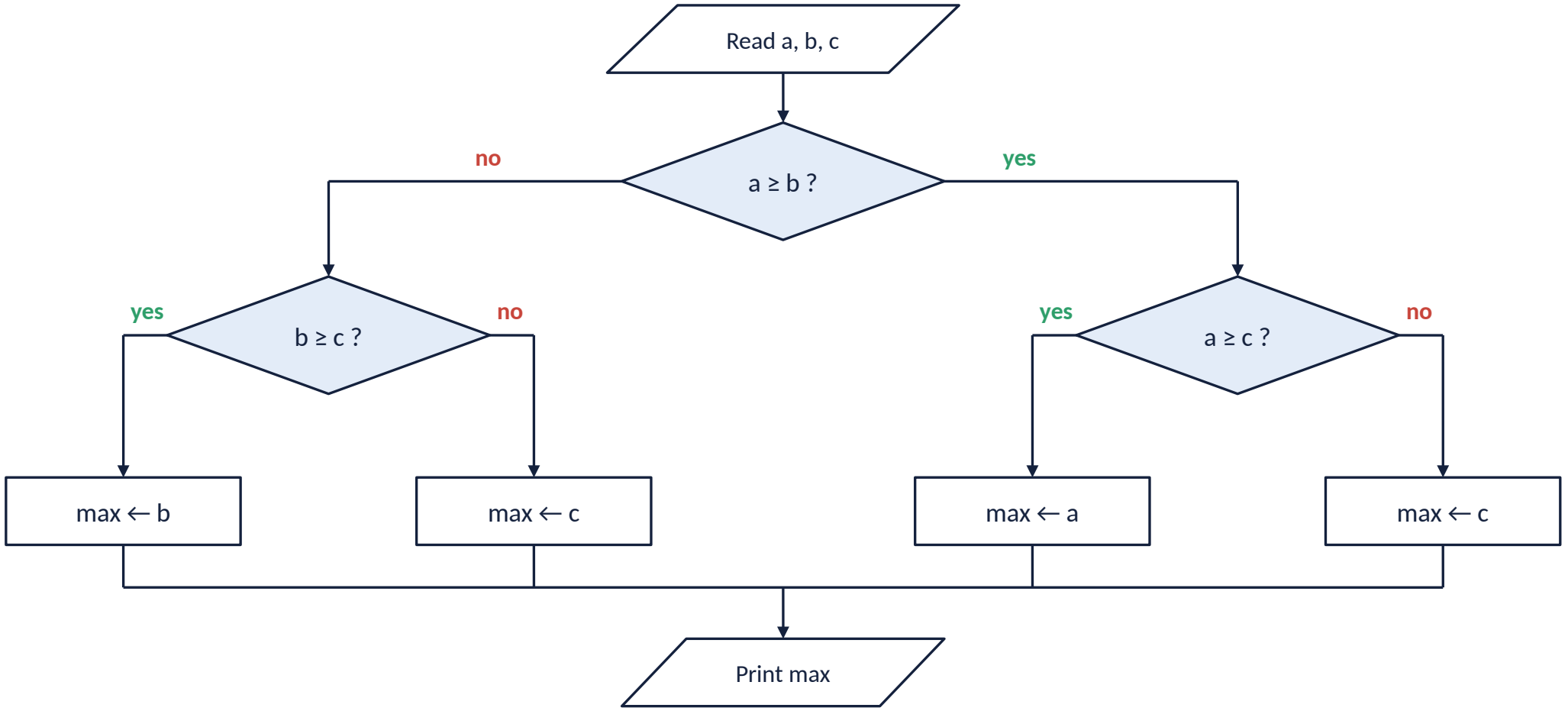
Program.cs (excerpt)

```
string letter;
if (score >= 90)
{
    letter = "AA";
}
else if (score >= 85)
{
    letter = "BA";
}
// ... BB, CB, CC, DC and DD in the same way ...
else
{
    letter = "FF";
}
```

Console

```
Enter the score (0-100): 85
Score 85 -> BA (grade point 3.5)
Ladder: BA   Switch expression: BA
```

Nested if: the Largest of Three Numbers



Largest of Three: Nested if vs. "Largest So Far"

Method 1: nested if-else

```
double max1;
if (a >= b)
{
    if (a >= c)
    {
        max1 = a;
    }
    else
    {
        max1 = c;
    }
}
else
{
    if (b >= c)
    {
        max1 = b;
    }
    else
    {
        max1 = c;
    }
}
```

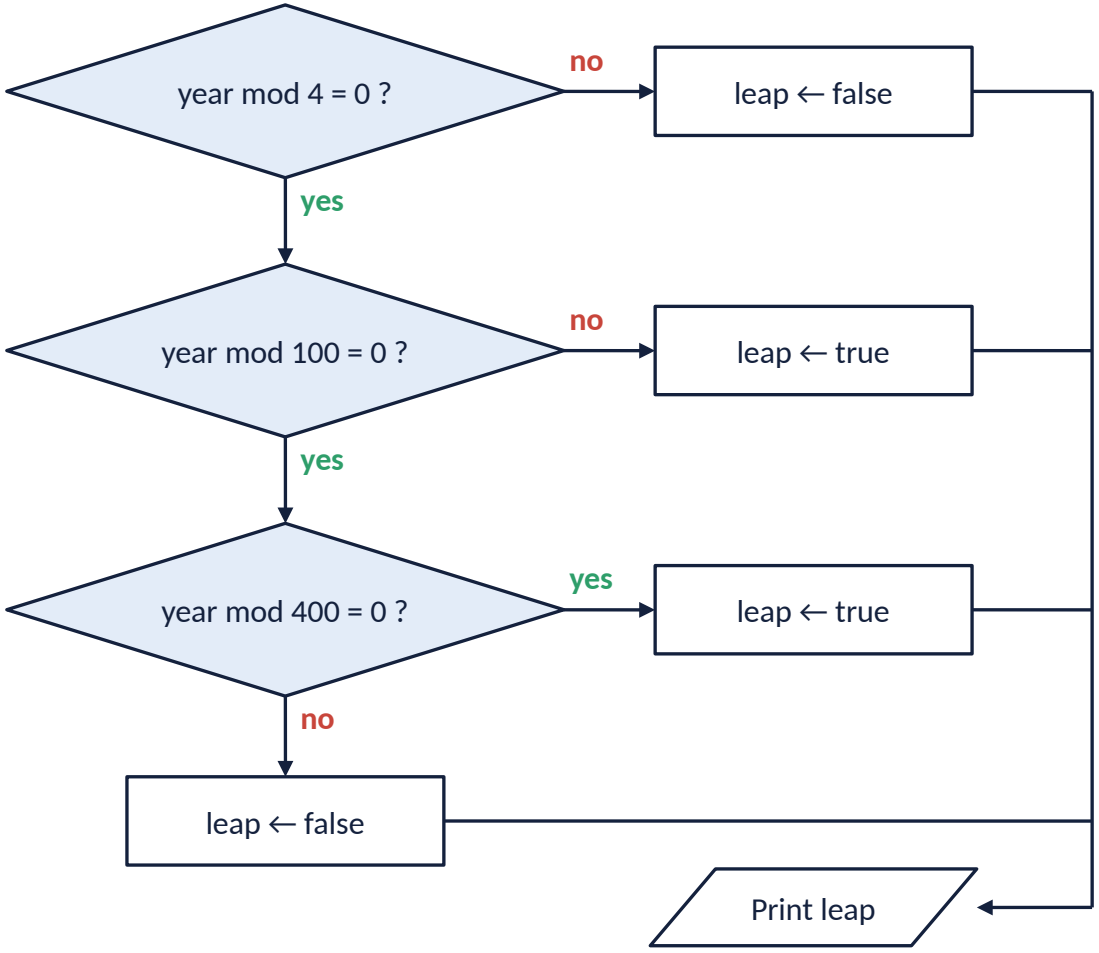
Method 2: largest so far

```
// Method 2: start with a, then let b and c challenge it.
double max2 = a;
if (b > max2)
{
    max2 = b;
}
if (c > max2)
{
    max2 = c;
}
```

Console

```
a = -2
b = -8
c = -5
Method 1 (nested if):      -2
Method 2 (largest so far): -2
```

Leap Year: Nested Decisions or One Expression




```
Method 2: one expression
bool leap2 = (year % 4 == 0 && year % 100 != 0)
             || year % 400 == 0;
```

Year	÷ 4	÷ 100	÷ 400	Leap?
2026	no	–	–	no
2024	yes	no	–	yes
1900	yes	yes	no	no
2000	yes	yes	yes	yes

```
Console
Enter a year: 1900
1900 is not a leap year (365 days).
Nested if: False   Single expression: False
```

The Dangling else: Which if Does It Belong To?

What does it print?

```
int x = 5;
int y = -3;
if (x > 0)
    if (y > 0)
        Console.WriteLine("both positive");
else
    Console.WriteLine("x is not positive");
Console.WriteLine("end");
```

Console

```
x is not positive
end
```



Rule: else belongs to the NEAREST if that has no else yet. The compiler ignores indentation.

Fixed with braces

```
if (x > 0)
{
    if (y > 0)
    {
        Console.WriteLine("both positive");
    }
}
else
{
    Console.WriteLine("x is not positive");
}
Console.WriteLine("end");
```

Console

```
end
```

>_ EXAMPLE 6 · 05_06_BMICATEGORY

Ranges on a double: BMI Categories

Underweight	Normal weight	Overweight	Obese
BMI < 18.5	18.5 ≤ BMI < 25	25 ≤ BMI < 30	BMI ≥ 30

```
bmi = weight / (height * height)
string category;
if (bmi < 18.5)
{
    category = "Underweight";
}
else if (bmi < 25)
{
    category = "Normal weight";
}
else if (bmi < 30)
{
    category = "Overweight";
}
else
{
    category = "Obese";
}
```

Console

```
Weight (kg): 72.5
Height (m): 1.78
BMI = 22.88 -> Normal weight

Weight (kg): 59.9
Height (m): 1.80
BMI = 18.49 -> Underweight

Weight (kg): 72.5
Height (m): 178
Error: weight 0-500 kg, height 0-3 m (e.g. 72.5, 1.78).
```

Triangle Type: Validate First, Then Classify

Program.cs (excerpt)

```
// Triangle inequality: each side < sum of the other two.
if (a + b <= c || a + c <= b || b + c <= a)
{
    Console.WriteLine("Not a triangle: one side is too long.");
    return;
}

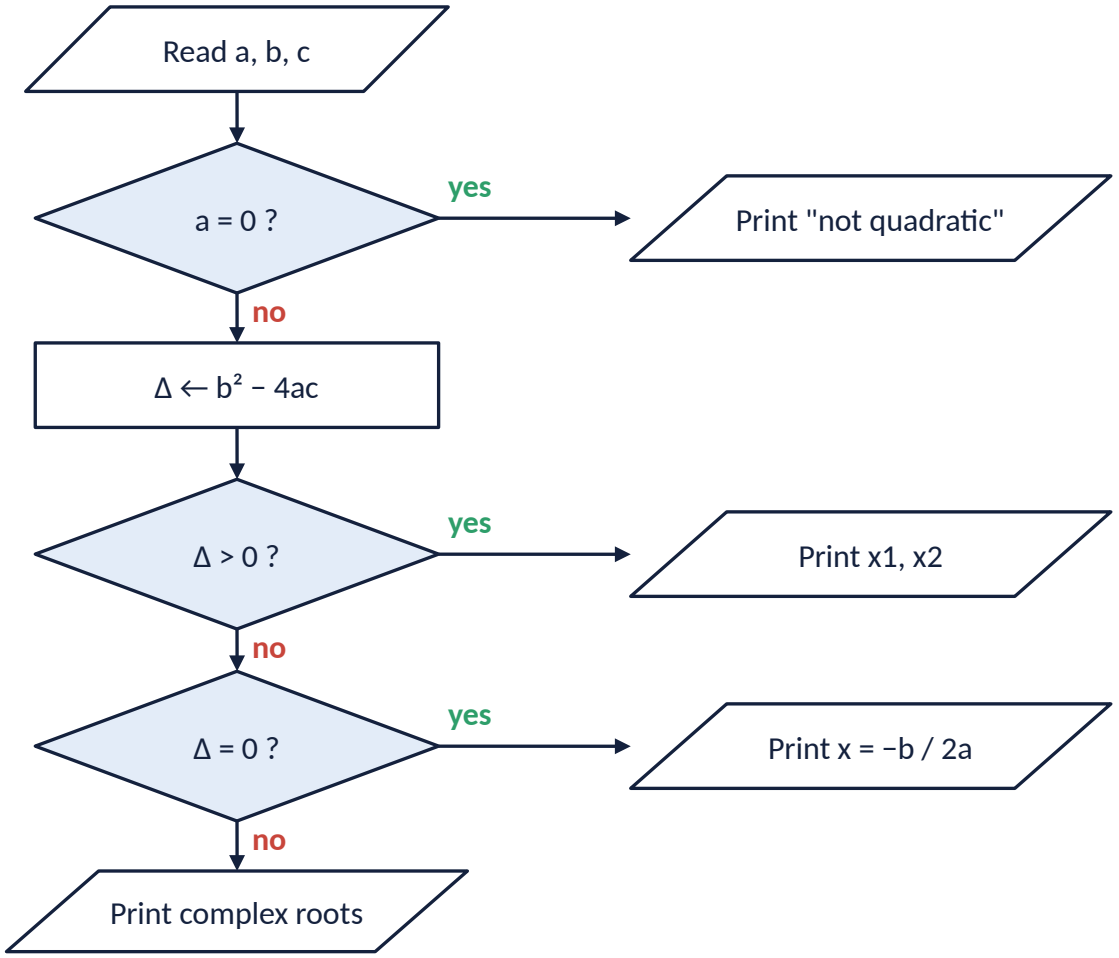
string type;
if (a == b && b == c)
{
    type = "equilateral";
}
else if (a == b || b == c || a == c)
{
    type = "isosceles";
}
else
{
    type = "scalene";
}
```

a b c	Output
5 5 5	equilateral
5 5 8	isosceles
8 5 5	isosceles
4 6 7	scalene
3 4 5	scalene + right triangle
1 2 3	Not a triangle
0 4 5	Error: side range



The right-angle test is an **independent if**: it is asked for every valid triangle.

Quadratic Equation: a Three-Way Decision



Discriminant $\Delta = b^2 - 4ac$

$\Delta > 0 \rightarrow$ two real roots $x = (-b \pm \sqrt{\Delta}) / 2a$
$\Delta = 0 \rightarrow$ one root $x = -b / 2a$
$\Delta < 0 \rightarrow$ no real root (complex pair)

Console

```
Solves a*x^2 + b*x + c = 0
a = 1
b = -3
c = 2
Discriminant = 1.00
Two real roots: x1 = 2.0000, x2 = 1.0000
```

Check a = 0 FIRST: otherwise we divide by 2a = 0.

>_ EXAMPLE 10 · CODE

Quadratic Equation in C#

Program.cs (excerpt)

```
double delta = b * b - 4 * a * c;
Console.WriteLine($"Discriminant = {delta:F2}");

if (delta > 0)
{
    double x1 = (-b + Math.Sqrt(delta)) / (2 * a);
    double x2 = (-b - Math.Sqrt(delta)) / (2 * a);
    Console.WriteLine($"Two real roots: x1 = {x1:F4}, x2 = {x2:F4}");
}
else if (delta == 0)
{
    double x = -b / (2 * a);
    Console.WriteLine($"One real (double) root: x = {x:F4}");
}
else
{
    double re = -b / (2 * a);
    double im = Math.Sqrt(-delta) / (2 * Math.Abs(a));
    Console.WriteLine($"Complex roots: {re:F4} + {im:F4}i and {re:F4} - {im:F4}i");
}
```

a b c	Output
1 −3 2	x1 = 2, x2 = 1
1 2 1	x = −1
1 2 5	−1 ± 2i
0 2 1	not quadratic

`delta == 0`
compares doubles exactly: fine for whole-number inputs; with decimals use a tolerance (Week 4).



PART 3

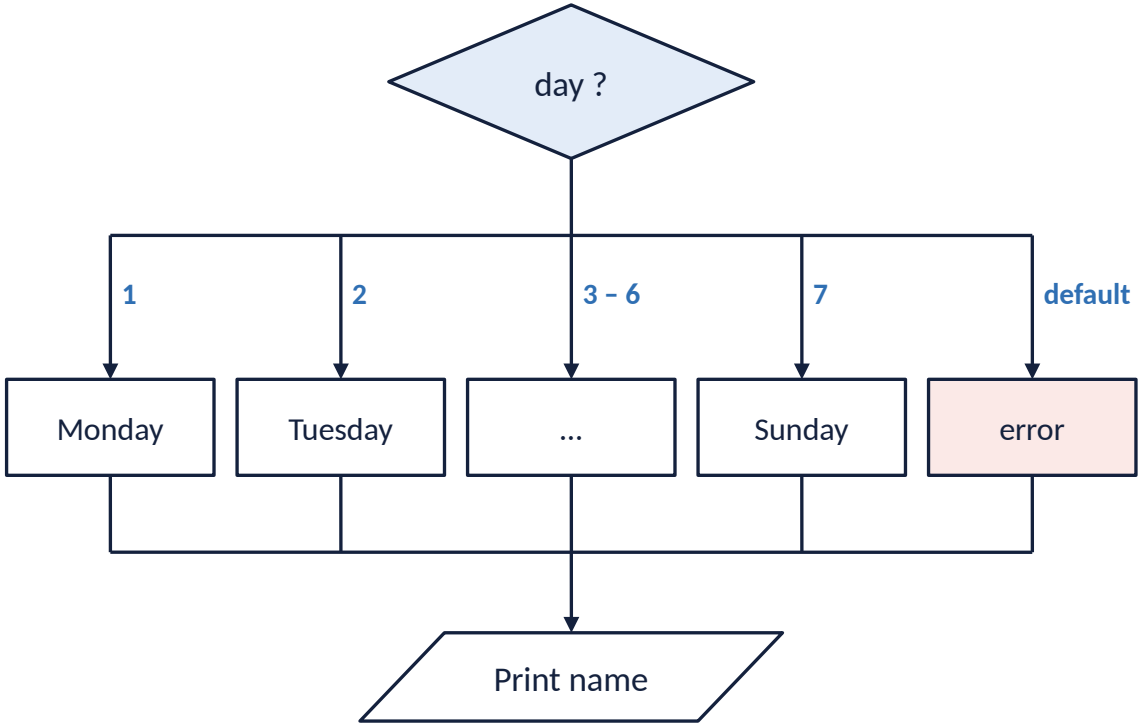
switch: Multi-Way Selection

One value, many cases

switch: Compare One Value with Many Cases

Syntax

```
switch (expression)
{
    case value1:
        // statements
        break;
    case value2:
    case value3:     // stacked labels
        // statements
        break;
    default:         // nothing matched
        // statements
        break;
}
```



Each exit is labelled with its case value; default catches everything else.

Case labels are **constants**: `1`, `'E'`, `"+"`.

Day Number → Day Name with switch

Program.cs (excerpt)

```
string name;
switch (day)
{
    case 1:
        name = "Monday";
        break;
    case 2:
        name = "Tuesday";
        break;
    // ... cases 3 to 6 in the same way ...
    case 7:
        name = "Sunday";
        break;
    default:
        Console.WriteLine("Error: the day must be 1-7.");
        return;   // return also ends a case
}
```

Console

```
Day number (1-7, Monday = 1): 3
Day 3 is Wednesday (weekday).

Day number (1-7, Monday = 1): 6
Day 6 is Saturday (weekend).

Day number (1-7, Monday = 1): 9
Error: the day must be 1-7.
```

Stacked labels: 6 and 7

```
switch (day)
{
    case 6:
    case 7:
        kind = "weekend";
        break;
    default:
        kind = "weekday";
        break;
}
```

No Fall-Through in C#: Every Section Must End

Forgot break (C / Java habit)

```
switch (day)
{
    case 6:
        Console.WriteLine("Saturday");
    case 7:
        Console.WriteLine("Sunday");
        break;
}
```

dotnet build

```
error CS0163: Control cannot fall through from one
case label ('case 6:') to another
```

Fix: add `break;` after "Saturday" (or stack the labels).

C# 9 patterns: or, and, relational

```
switch (day)
{
    case 6 or 7:
        Console.WriteLine("weekend");
        break;
    case >= 1 and <= 5:
        Console.WriteLine("weekday");
        break;
    default:
        Console.WriteLine("invalid");
        break;
}
```

A section ends with `break`, `return`, `throw` or `goto case`.

Calculator: switch on a string

Program.cs (excerpt)

```
switch (op)
{
    // cases "+", "-" and "%": see the code folder
    case "*":
    case "x":   // stacked labels: *, x and X share one block
    case "X":
        Console.WriteLine($"{a} * {b} = {a * b:F2}");
        break;
    case "/":
        if (b == 0)
        {
            Console.WriteLine("Error: cannot divide by zero.");
        }
        else
        {
            Console.WriteLine($"{a} / {b} = {a / b:F2}");
        }
        break;
    default:
        Console.WriteLine($"Error: unknown operator \"{op}\".");
        break;
}
```

Console

```
First number: 7
Operator (+ - * / %): /
Second number: 2
7 / 2 = 3.50

First number: 7
Operator (+ - * / %): /
Second number: 0
Error: cannot divide by zero.

First number: 7
Operator (+ - * / %): ^
Second number: 2
Error: unknown operator "^".
```



double `7 / 0` is **Infinity**, not an error — check `b == 0` yourself.

Shipping Cost: Brackets + switch on a char

Weight w (kg)	Base cost (TL)
0 < w ≤ 1	60
1 < w ≤ 5	90
5 < w ≤ 20	150
20 < w ≤ 100	150 + 8 × ⌈w − 20⌉

Express: base cost × 1.5

Input	Total (real output)
`1 S`	60.00 TL
`1.01 s`	90.00 TL
`20 E`	225.00 TL
`20.1 S`	158.00 TL
`2 X`	Error: unknown service 'X'.

Program.cs (excerpt)

```
switch (service)
{
    case 'S':
    case 's':
        serviceName = "standard";
        total = baseCost;
        break;
    case 'E':
    case 'e':
        serviceName = "express";
        total = baseCost * ExpressFactor;
        break;
    default:
        Console.WriteLine($"Error: unknown service '{service}'.");
        return;
}
```

Console

```
Parcel weight (kg, max 100): 23.5
Service (S = standard, E = express): e
Base cost: 182.00 TL
Total (express): 273.00 TL
```

The switch Expression (C# 8) with Relational Patterns (C# 9)

Letter grade (05_05_LetterGrade)

```
string letter2 = score switch
{
    >= 90 => "AA",
    >= 85 => "BA",
    >= 80 => "BB",
    >= 75 => "CB",
    >= 70 => "CC",
    >= 65 => "DC",
    >= 60 => "DD",
    _ => "FF"
};
```

Console

```
Enter the score (0-100): 85
Score 85 -> BA (grade point 3.5)
Ladder: BA   Switch expression: BA
```

Produces a value

Assign it: string letter2 = score switch { … };

Relational patterns

>= 90, < 18.5 — combined with and / or (C# 9).

_ is the default arm

Without it: warning CS8509 and a possible crash.

Order is checked

An arm that can never match is error CS8510.

Which Decision Structure When?

Structure	Chooses	Best for	Example
if	0 or 1 block	an optional extra step	`if (x < 0) { x = -x; }`
if-else	1 of 2 blocks	two alternatives	`PASS / FAIL`
else-if ladder	1 of many blocks	ranges, mixed conditions	`grades, BMI, prices`
nested if	paths of a decision tree	a question depends on an answer	`max of three, leap year`
switch statement	1 of many sections	exact values of one variable	`day number, operator`
switch expression	1 of many values	value from ranges / values	`score switch { >= 90 => "AA", … }`
?:	1 of 2 values	short two-way value	`n % 2 == 0 ? "even" : "odd"`



Choose the clearest one. Anything can be written with if-else alone.



PART 4

Input Validation and Testing

Never trust the keyboard — test every branch

int.TryParse: Validate Instead of Crashing

Guard-clause pattern (05_02_EvenOdd)

```
Console.Write("Enter an integer: ");
if (!int.TryParse(Console.ReadLine(), out int n))
{
    Console.WriteLine("Error: that is not a whole number.");
    return;   // ends the program here (a "guard clause")
}
```

int.Parse("abc") without validation

```
Unhandled exception. System.FormatException: The input string
'abc' was not in a correct format.
```

`out int n`: TryParse writes the number into n and returns true / false.

Call	Returns	Value
int.TryParse("42", …)	true	42
int.TryParse(" 42 ", …)	true	42
int.TryParse("+7", …)	true	7
int.TryParse("3.5", …)	false	0
int.TryParse("", …)	false	0
int.TryParse("99999999999", …)	false	0
double.TryParse("72,5", …)	**true**	**725 (!)**
double.TryParse("NaN", …)	true	NaN

With the invariant culture the comma is a thousands separator: a range check must catch it.

Pass or Fail — with Validation

Program.cs (excerpt)

```
// Each test runs only if all the tests above it were false.
if (!int.TryParse(input, out int score))
{
    Console.WriteLine("Error: please enter a whole number.");
}
else if (score < 0 || score > 100)
{
    Console.WriteLine("Error: the score must be 0-100.");
}
else if (score >= PassMark)
{
    Console.WriteLine($"Score {score}: PASS");
}
else
{
    Console.WriteLine($"Score {score}: FAIL");
}
```

Console

```
Enter the exam score (0-100): abc
Error: please enter a whole number.

Enter the exam score (0-100): 101
Error: the score must be 0-100.

Enter the exam score (0-100): 60
Score 60: PASS

Enter the exam score (0-100): 59
Score 59: FAIL
```

Order of the checks

1 Is it a whole number?
2 Is it in the range 0–100?
3 Pass or fail?

ATM Withdrawal: the Order of the Rules

Program.cs (excerpt)

```
if (amount <= 0)
{
    Console.WriteLine("Refused: amount must be positive.");
}
else if (amount % 10 != 0)
{
    Console.WriteLine("Refused: only multiples of 10 TL.");
}
else if (amount > balance)
{
    Console.WriteLine("Refused: insufficient funds.");
}
else
{
    balance -= amount;
    Console.WriteLine($"Please take your {amount:F2} TL.");
    Console.WriteLine($"New balance: {balance:F2} TL");
}
```

Balance	Amount	Result (real output)
500	200	paid, new balance 300.00
500	500	paid, new balance 0.00
500	510	insufficient funds
500	25	only multiples of 10 TL
500	0	amount must be positive
500	-10	amount must be positive

Console

```
Account balance (TL): 500
Amount to withdraw (TL): 200
Please take your 200.00 TL.
New balance: 300.00 TL
```

Login Check: Comparing Strings

Program.cs (excerpt)

```
const string CorrectUser = "student";
const string CorrectPassword = "Okan2026!";

Console.Write("Username: ");
string user = Console.ReadLine();
Console.Write("Password: ");
string password = Console.ReadLine();

// == compares strings exactly: "Student" is not "student".
if (user == "" || password == "")
{
    Console.WriteLine("Error: empty username or password.");
}
else if (user == CorrectUser && password == CorrectPassword)
{
    Console.WriteLine($"Login successful. Welcome, {user}!");
}
else
{
    // Do not say WHICH part was wrong: it helps attackers.
    Console.WriteLine("Wrong username or password.");
}
```

Console

```
Username: student
Password: Okan2026!
Login successful. Welcome, student!

Username: Student
Password: Okan2026!
Wrong username or password.
```



Security habits

Never reveal which part was wrong. Real systems never keep passwords in the source code.

Boundary Testing: One Test per Branch, Two per Border



Test	Input	Expected	Actual output (05_01_PassFail)	OK
not a number	`abc`	error message	`Error: please enter a whole number.`	✓
just below range	`-1`	error message	`Error: the score must be 0-100.`	✓
lower border	`0`	FAIL	`Score 0: FAIL`	✓
just below pass mark	`59`	FAIL	`Score 59: FAIL`	✓
pass mark	`60`	PASS	`Score 60: PASS`	✓
upper border	`100`	PASS	`Score 100: PASS`	✓
just above range	`101`	error message	`Error: the score must be 0-100.`	✓

>_ COMMON MISTAKES

Six Classic Decision Bugs

= instead of ==

`if (x = 5)` → error CS0029 (int is not bool). Write `x == 5`.

Chained comparison

`0 <= x <= 100` → error CS0019. Write `x >= 0 && x <= 100`.

Stray semicolon

`if (t > 30);` ends the if: the block below always runs (warning CS0642).

Missing braces

Only the first statement belongs to the if; an else pairs with the nearest if.

Missing break

Each case section must end with `break` or `return` (error CS0163).

Wrong ladder order

`>= 60` before `>= 90`: AA is never reached. Test the borders!

What Does Each Snippet Print?

Snippet A

```
int a = 7, b = 3;
if (a > 5 && b > 5)
    Console.WriteLine("A");
else if (a > 5 || b > 5)
    Console.WriteLine("B");
else
    Console.WriteLine("C");
```

Snippet B

```
int temperature = 15;
if (temperature > 30);
{
    Console.WriteLine("It is hot!");
}
Console.WriteLine("Done.");
```

Snippet C

```
int x = 5;
int y = -3;
if (x > 0)
    if (y > 0)
        Console.WriteLine("both positive");
else
    Console.WriteLine("x is not positive");
Console.WriteLine("end");
```

Snippet D

```
int n = 15;
if (n % 3 == 0) Console.WriteLine("Fizz");
if (n % 5 == 0) Console.WriteLine("Buzz");
if (n % 3 != 0 && n % 5 != 0) Console.WriteLine(n);
```

switch and switch Expressions: Predict or Fix

Snippet E — and for k = 1? k = 5?

```
int k = 2;
switch (k)
{
    case 1:
    case 2:
        Console.WriteLine("small");
        break;
    default:
        Console.WriteLine("big");
        break;
}
```

Snippet F

```
int t = 25;
string msg = t switch
{
    < 0 => "freezing",
    < 15 => "cold",
    < 25 => "mild",
    _ => "warm"
};
Console.WriteLine(msg);
```

Snippet G — find the bug

```
switch (day)
{
    case 6:
        Console.WriteLine("Saturday");
    case 7:
        Console.WriteLine("Sunday");
        break;
}
```

Your turn

Write the output of E and F, and explain what the compiler says about G.

Answers to Quiz 1 and Quiz 2

	Output	Why
A	`B`	a > 5 && b > 5 is false (b is 3); a > 5 || b > 5 is true.
B	`It is hot!  /  Done.`	The ; ends the if; the { } block is independent and always runs.
C	`x is not positive  /  end`	The else belongs to the nearest if: if (y > 0).
D	`Fizz  /  Buzz`	Three independent ifs, not a ladder: two tests are true.
E	`small`	k = 2 matches the stacked labels 1 and 2. k = 1 → small, k = 5 → big.
F	`warm`	25 < 25 is false, so only the discard arm _ matches.
G	`does not compile`	error CS0163: add break; after "Saturday".

>_ PRACTICE · IN CLASS

In-Class Exercises (Pairs)

E1 ★

Sign of a number

Read an integer; print positive, negative or zero.

E2 ★

Vowel or consonant

Read one letter (char.TryParse); switch with stacked labels a e i o u, both cases.

E3 ★★

Days in a month

Month 1–12 and year → 28, 29, 30 or 31 days (stacked labels + leap rule).

E4 ★★

Weather advice

Temperature → advice with a switch expression and relational patterns.

E5 ★★

Electricity bill

First 150 kWh at 2.00 TL, the rest at 3.00 TL; validate kWh ≥ 0.

E6 ★★★

Rock–paper–scissors

Two players type R, P or S; print the winner or a draw.

Homework (before Week 6)

H1 Parking fee	First hour 40 TL, each further started hour 25 TL, at most 250 TL per day. Read the minutes parked.
H2 Min and max of three	Print the smallest and the largest of three numbers, or "all equal".
H3 Flowchart by hand	Draw the complete flowchart of Example 9 (triangle type), including validation.
H4 Test plan	Write a boundary test table for 05_13_ShippingCost, run every test and record the output.
H5 Unit converter	Menu 1–4 (km→mile, mile→km, kg→lb, lb→kg) with switch; validate every input.

>_ SUMMARY

Key Takeaways

01 if runs a block only when a bool condition is true; if-else chooses exactly one of two blocks.

02 In an else-if ladder the FIRST true condition wins — order the conditions carefully.

03 Always use braces: they prevent the missing-brace, stray-semicolon and dangling-else bugs.

04 switch compares one value with constant labels; every section ends with break (no fall-through).

05 switch expressions and ?: choose a VALUE; if-else and switch statements choose ACTIONS.

06 Validate input with TryParse + range checks, and test every branch at its borders.

NEXT WEEK

Loops: the for Loop



MCMP113
Course materials